

Evaluating Common AI APIs and Models for Logan

A comparative analysis of frontier and value-tier language models

As of May 9, 2026

Executive Summary

Top Value Pick: Grok 4.3

As of May 9, 2026, the strongest high-confidence value play in the retrieved data is Grok 4.3 on the first-party path. It is priced at \$1.25 per million input tokens and \$2.50 per million output tokens, supports a 1 million token context window, and Artificial Analysis reports an Intelligence Index score of 53 with first-token latency around 0.75 seconds on xAI's own endpoint. On a simple 75/25 input/output blended-cost proxy, that works out to about \$1.56 per million blended tokens and roughly \$0.029 per quality point, which is dramatically cheaper than the flagship premium tier from either OpenAI or Anthropic while still benchmarking in the frontier band.

If maximum answer quality matters more than spend, the premium leaders in the retrieved benchmark set are GPT-5.5 and Claude Opus 4.7. Artificial Analysis places GPT-5.5 at 60 on its Intelligence Index and Claude Opus 4.7 at 57. Their official first-party short-context prices are \$5/\$30 and \$5/\$25 per million input/output tokens respectively, which makes them excellent escalation models rather than ideal defaults for routine work.

For a practical student-plus-technical workflow, the strongest operating pattern is a three-lane stack. Use GPT-5.4 mini as the main OpenAI workhorse for tool-heavy tasks and fast iteration, Grok 4.3 as the frontier-value challenger/checker, and Claude 4.5 Haiku or Gemini 2.5 Flash for high-volume drafts, extraction, classification, RAG, and batch jobs. Then reserve GPT-5.5 or Claude Opus 4.7 for the hardest 10–20% of prompts where quality gaps matter. That mix is the best fit for someone who already has keys and budgets loaded and wants to control spend without giving up a premium backstop.

Cheapest General Text Options

The cheapest general text options visible in the retrieved first-party material were Cohere Command R7B at \$0.0375/\$0.15 per million input/output tokens, Gemini 2.5 Flash-Lite at \$0.10/\$0.40 on Vertex pricing, OpenAI GPT-5.4 nano at \$0.20/\$1.25, and Azure OpenAI GPT-4.1-nano at \$0.10/\$0.40 on the global tier. Those are not interchangeable with frontier models, but they are the right bill-reducers for bulk transformations, prefilters, routers, and “good enough” first passes.

Method and Normalization

This report focuses on mainstream first-party text and multimodal generation APIs rather than embeddings, rerankers, media generation, or speech-specific SKUs, because that is the cleanest common denominator across the providers you named. Model lists and prices come from first-party docs whenever possible. For cross-provider quality and latency, I use benchmark summaries from Artificial Analysis where official docs do not expose directly comparable metrics.

Blended Conversational Cost Proxy

I normalize price with a simple proxy: $0.75 \times \text{input price} + 0.25 \times \text{output price}$, all in USD per 1 million tokens, using the standard first-party tier and the lower context-price band where providers publish different prices for short and long prompts. This is not a universal law; it is just a sane default when you did not specify a tighter workload ratio. When a provider offers batch/flex pricing, I treat that as a discount lever rather than the default production price.

Quality is shown in two ways. Where the retrieved benchmark sources exposed a current numeric score, I report the Artificial Analysis Intelligence Index directly. Where a current numeric score was not exposed in the retrieved snippets, I keep the judgment qualitative and say so explicitly rather than invent a number. Latency-adjusted value is only computed where both benchmark and latency figures were actually visible in retrieved sources.

Provider Inventory

OpenAI

The current public frontier family centers on GPT-5.5, GPT-5.4, GPT-5.4 mini, GPT-5.4 nano, and higher-priced Pro variants. The model docs explicitly recommend GPT-5.5 for complex reasoning and coding, and GPT-5.4 mini or nano for lower-latency, lower-cost workloads. OpenAI's pricing page also exposes Standard, Batch, Flex, and Priority service tiers; Batch and Flex are effectively half-price relative to Standard for the GPT-5.5 / GPT-5.4 family, while Priority is a premium latency tier. Rate limits are model- and tier-dependent and auto-increase with spend.

Anthropic

Current primary models are Claude Opus 4.7, Claude Sonnet 4.6, and Claude Haiku 4.5. Anthropic's official comparison page positions Opus 4.7 as the most capable general-availability model, Sonnet 4.6 as the speed/intelligence balance, and Haiku 4.5 as the fastest model with near-frontier intelligence. Anthropic also offers batch processing at a 50% token discount, prompt caching with 0.1× cache-hit pricing, a 1.1× US-only inference multiplier for newer models, and enterprise volume discounts. New users receive a small amount of free credits.

xAI

xAI's docs now strongly recommend Grok 4.3 as the default chat/coding choice. It supports configurable reasoning and a 1M-token context. The same docs show older Grok 4.20 variants and Grok 4.1 Fast variants, but xAI also states that several older models, including Grok 4.1 Fast and Grok Code Fast 1, retire on May 15, 2026. Batch API pricing is 20% to 50% below standard pricing, spend-based rate-limit tiers unlock automatically, and enterprise customers can buy provisioned throughput with a minimum 30-day commitment for predictable capacity.

Google (AI Studio and Vertex)

Google's current first-party lineup is centered on Gemini 2.5 Pro, Gemini 2.5 Flash, and Gemini 2.5 Flash-Lite, with Gemini 3.1 live/TTS and image/video preview families appearing on the same docs pages. Gemini AI Studio exposes free and paid tiers. Pro, Flash, and Flash-Lite all have Standard, Batch/Flex, and Priority pricing modes in the official docs, and Google's rate-limit docs show project-level RPM/TPM/RPD tiers plus very large batch queues. On Vertex, Google also publishes Standard PayGo throughput tiers and Provisioned Throughput as a fixed-cost reserved-capacity product.

Microsoft Azure and Azure OpenAI

The retrieved Azure pricing and quota material shows GPT-4.1, GPT-4.1 mini, and GPT-4.1 nano still as prominent Azure OpenAI workhorses, with newer GPT-5.x entries also appearing on the pricing page. Microsoft's quota docs show why Azure is attractive operationally: GPT-4.1 mini and nano have far higher published RPM/TPM ceilings than full GPT-4.1 on GlobalStandard deployments, and Microsoft also offers PTU-based provisioned throughput billed hourly with reservation discounts. Azure is best understood as a compliance/governance/throughput channel for OpenAI-family models rather than a distinct model family.

Cohere

Cohere still exposes Command A, Command A Reasoning, Command R7B, Command R, and Command R+ in the docs. Command A is the premium general-purpose enterprise model at \$2.5/\$10 per million input/output tokens; Command A Reasoning is free through rate limits on trial and production keys and becomes a production purchase through Model Vault; Command R7B is the ultra-cheap latency-sensitive option. Trial/evaluation keys are free but capped, newer models often require sales contact for full production limits, and Model Vault adds hourly private deployment pricing with discounts for longer commitments.

Mistral AI

Mistral's current featured lineup includes Mistral Medium 3.5, Mistral Large 3, Devstral 2, Mistral Small 4, Codestral, and the Ministral 3 family. Official model cards show very aggressive pricing for several open-weight models, including Mistral Large 3 at \$0.5/\$1.5 per million input/output tokens, Devstral 2 at \$0.4/\$2, Codestral at \$0.3/\$0.9, and Ministral 3 14B at \$0.2/\$0.2. Mistral is the most interesting "open-weight but still API-usable" provider in this set if you care about eventual self-hosting optionality.

Comparison Table

The table below is sorted for practical evaluation order rather than alphabetical order. “Blend \$/M” uses the 75% input / 25% output proxy described in the Method section. “AA score” appears only where the retrieved benchmark material exposed a current score clearly enough to use without guessing. NR = not reported in retrieved snippets. Green-highlighted rows indicate recommended value picks.

Model	Provider	Type	Ctx	In \$/M	Out \$/M	Blend	Benchmark	Latency	Best Use
Grok 4.3	xAI	Flagship configurable reasoning LLM	1M	\$1.25	\$2.50	\$1.56	AA 53	0.75s TTFT 69 t/s	Frontier at near-mid-tier prices
GPT-5.4 mini	OpenAI	Strong mini multimodal reasoning LLM	400k	\$0.75	\$4.50	\$1.69	High-ranked (score NR)	0.76s TTFT 154 t/s	Best OpenAI daily default
Claude 4.5 Haiku	Anthropic	Fast near-frontier LLM	200k	\$1.00	\$5.00	\$2.00	AA 31	0.69s TTFT 95 t/s	Bulk chat, RAG, extraction
Gemini 2.5 Flash	Google	Hybrid reasoning multimodal LLM	1M	\$0.30	\$2.50	\$0.85	Balance d (NR)	High quota/ batch cap	Cheap long-context general
Gemini 2.5 Flash-Lite	Google	Cheapest Gemini 2.5 text model	1M	\$0.10	\$0.40	\$0.18	Budget tier	High throughput	Bulk triage, classification
GPT-5.5	OpenAI	Flagship multimodal reasoning LLM	1M	\$5.00	\$30.00	\$11.25	AA 60	Fast	Hardest coding, analysis
Claude Opus 4.7	Anthropic	Flagship reasoning/coding LLM	1M	\$5.00	\$25.00	\$10.00	AA 57	Moderate 19.4 t/s	Highest-quality Anthropic path
Gemini 2.5 Pro	Google	Frontier reasoning multimodal LLM	1M	\$1.25	\$10.00	\$3.44	Frontier (NR)	125 t/s AI Studio	Long-context, grounded work
Command A	Cohere	Enterprise/RAG/tool-use LLM	256k	\$2.50	\$10.00	\$4.38	Enterprise	2.4–2.8s TTFT 39 t/s	Multilingual RAG, agents
Command R7B	Cohere	Small utility LLM	128k	\$0.04	\$0.15	\$0.07	Small/utility	Latency-sensitive	Cheapest bulk text work
Mistral Medium 3.5	Mistral AI	Frontier-class multimodal LLM	256k	\$1.50	\$7.50	\$3.00	AA 39	NR	Agentic, coding, open-weight
Mistral Large 3	Mistral AI	Open-weight general multimodal LLM	256k	\$0.50	\$1.50	\$0.75	Open-weight flagship	—	Low-cost, self-hosting option

Provider-Level Value Picks

A few provider-specific “cheapest vs best value” calls are unusually clear. Within OpenAI, GPT-5.4 nano is the cheapest listed text model, but GPT-5.4 mini is the more useful default for real work. Within Anthropic, Claude 4.5 Haiku is both the cheapest and the practical value leader. Within xAI, Grok 4.1 Fast is cheaper but retiring; among current recommended models, Grok 4.3 is the value leader. Within Google Gemini, Flash-Lite is the price floor and Flash is the general-value sweet spot. Within Cohere, Command R7B is the cost floor and Command A is the enterprise-quality choice. Within Mistral AI, the docs emphasize the low-cost Ministral family and aggressively priced Large 3 / Devstral / Codestral cards for differentiated workloads.

Charts

Chart 1 — Cost vs. Quality

The first chart uses only the subset where enough confidence exists to put price and benchmark quality on one plane without inventing missing numbers. It shows why Grok 4.3 is the standout “value frontier” point in the current retrieved dataset, while GPT-5.5 and Claude Opus 4.7 remain premium-quality, premium-cost choices.



Figure 1. Blended cost (x-axis) vs. Artificial Analysis Intelligence Index quality (y-axis) for models with confirmed numeric scores. Best value quadrant = low cost, high quality.

Chart 2 — Blended Conversational Cost per 1M Tokens

The second chart is a straight price view. For bulk work, the order that matters most is Command R7B, Gemini 2.5 Flash-Lite, Mistral Large 3, Gemini 2.5 Flash, Grok 4.3 / GPT-5.4 mini, then the premium flagships.

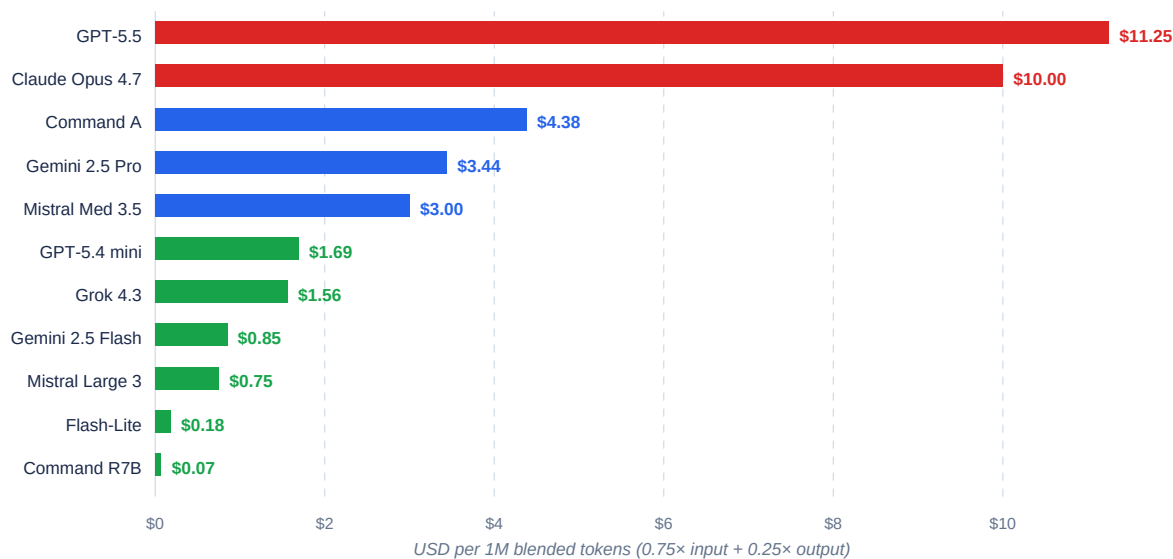


Figure 2. Horizontal bar chart of blended cost. Green < \$2/M, blue < \$5/M, amber < \$8/M, red ≥ \$8/M.

Chart 3 — Vendor Selection Decision Tree

For actual vendor choice, the decision tree is simpler than the market noise suggests: start with a strong cheap default, keep one higher-end escalator, and split “bulk transforms” from “hard reasoning.” That is the right shape unless you have strict compliance, region, or hosted-throughput constraints.

Decision Criterion	Branch	Recommendation
Need maximum answer quality?	Yes →	Use GPT-5.5 or Claude Opus 4.7
Need best frontier value?	Yes →	Use Grok 4.3
Need cheapest high-volume processing?	Yes →	Use Gemini 2.5 Flash-Lite or Command R7B
Need OpenAI tools and ecosystem?	Yes →	Use GPT-5.4 mini
Need low-cost long context or grounding?	Yes →	Use Gemini 2.5 Flash
Need multilingual enterprise RAG?	Yes →	Use Command A or Claude 4.5 Haiku
None of the above	→	Run your own bake-off on real prompts

Figure 3. Evaluate criteria top-to-bottom; stop at the first Yes branch that applies to your workload.

Recommendations and Test Prompts

#1

Grok 4.3 — First Model to Benchmark Seriously

For your use case, my top recommendation is Grok 4.3 as the first model to benchmark seriously. The reason is simple: in the retrieved evidence, it is the cleanest combination of frontier-level benchmark position, low official price, large context, and very strong measured latency. That makes it the best “value frontier” baseline to test against everything else you already fund.

#2

GPT-5.4 mini — Likely Everyday Default on the OpenAI Side

My second recommendation is GPT-5.4 mini as your likely everyday default on the OpenAI side. OpenAI itself positions it as the strongest mini model, and the retrieved performance snippets show very fast output with sub-second first-token latency territory. If you are doing tool use, coding loops, structured outputs, or general assistant work where ecosystem polish matters, this is probably the best “already-good-enough” OpenAI default before you escalate to GPT-5.5.

#3

Claude 4.5 Haiku — Cheap Fast Volume on Anthropic

My third recommendation is Claude 4.5 Haiku for cheap fast volume on Anthropic. The official docs and the benchmark provider page both point the same way: it is the fastest Anthropic model, the lowest-latency Anthropic model in the retrieved data, and the cheapest serious Claude model by a wide margin. For drafts, summarization, extraction, and many RAG paths, it is the Claude SKU that is hardest to argue against economically.

#4

GPT-5.5 — Premium Escalator

My fourth recommendation is GPT-5.5 as your premium escalator. You do not want to run everything through it, but you do want it available for the hardest tasks: messy codebases, high-stakes reasoning, professional writing, and difficult synthesis. The retrieved benchmark material puts it at the top of the Artificial Analysis leaderboard, and OpenAI’s own docs put it forward as the starting point for complex reasoning and coding.

#5

Gemini 2.5 Flash — Most Attractive Google Default

My fifth recommendation is Gemini 2.5 Flash as the most attractive Google default for low-cost long-context work. The official pricing is very low for a 1M-context model, Google gives you batch/flex options plus free tier plus grounding, and Vertex adds serious throughput paths if you later need enterprise scaling. If your real workload involves lots of long documents, retrieval, grounded comparisons, or Google-native deployment, Flash is the Google model I would put into the first bake-off.

Evaluation Harness

A sensible evaluation harness for your own keys should log five things on every run: total wall time, time to first token, tokens per second after first token, exact token bill from the provider response, and a task-specific pass/fail score. Without that, people end up comparing list prices rather than comparing actual job cost. That mistake is one reason model spend drifts. For Logan’s workflow, test on tasks that look like real work: long-context extraction from manuals or class material, coding/debugging, strict JSON tool use, adversarial factuality, and concise answer quality under token limits.

Recommended Test Prompt Bundle

A strong prompt bundle to run with your current keys would be this:

#	Test Family	Prompt	What It Tests
1	Long-Context Extraction	Read the attached 40–100 page document and extract the exact torque specs, safety warnings, and service intervals into strict JSON. Quote page-local evidence for every field.	Tests long context, extraction fidelity, hallucination control, and structured output.
2	Coding / Debugging	Given this failing Python or JavaScript unit test file plus source file, fix the bug with the smallest patch, explain the root cause in less than 120 words, and output a unified diff only.	Tests coding quality, patch discipline, and verbosity control.
3	Tool-Use / Schema Compliance	You may call tools. Return only valid JSON matching this schema. If you cannot support a field with evidence, emit null and explain the missing evidence in a single short uncertainty field.	Tests tool-use discipline and schema compliance.
4	Calibrated Uncertainty	Answer the question, then give a confidence estimate from 0 to 1 and one sentence naming the exact fact that would most likely falsify your answer.	Tests calibrated uncertainty instead of confident bluffing.
5	Compression / Instruction-Following	Summarize this forum thread or lecture transcript into a two-paragraph executive brief, one paragraph of disagreements, and one follow-up question list, all under 220 tokens total.	Tests compression quality and instruction-following under tight output budgets.

Minimal Benchmark Sheet Columns

If you want a minimal benchmark sheet, use these columns: provider, model, prompt family, input tokens, output tokens, wall time, TTFT, sustained t/s, quoted cost, schema-valid yes/no, factual pass/fail, citation-grounded yes/no, and human preference vote. That sheet will tell you more in one afternoon than another week of reading model marketing pages.

Open Questions and Limitations

Gap 1: Uniform Benchmark Coverage

The largest unresolved gap is uniform benchmark coverage. I was able to retrieve clean numeric benchmark scores for GPT-5.5, Claude Opus 4.7, Claude 4.5 Haiku, Grok 4.3, and Mistral Medium 3.5, but not for every current stable Google, OpenAI-mini, Cohere, and Mistral model in the same accessible snippet format. I therefore kept some rows qualitative rather than pretending to have a number I could not validate.

Gap 2: Azure Normalization

The second gap is Azure normalization. Microsoft's official pricing page is highly dynamic and region-sensitive in the retrieved HTML, so I could confirm some prices and quotas cleanly, especially around GPT-4.1 mini and nano, but not build a fully normalized Azure model-by-model table with the same confidence as the direct first-party providers. The broad conclusion is still clear: Azure is the right route when you care about enterprise hosting, regional controls, high throughput, and PTU reservations more than pure first-party simplicity.

Gap 3: Actual Prompt Mix

The third gap is your actual prompt mix. All cost-efficiency rankings change if your workload is, for example, 95% input-heavy long-context retrieval, or 70% code generation, or dominated by high output-token tasks. The blended proxy used throughout this report is a starting-point assumption, not a guaranteed fit.

Coding AI API Addendum

Code-focused supplement to the main AI API evaluation. Covers cheapest code-capable APIs, provider-by-provider coding value picks, prompt caching strategy for image-heavy debugging workflows, and a recommended coding bake-off harness.

Executive Summary

Key Shift: Cheapest $\neq$ Most Useful for Coding

For a code-focused addendum, the most important change from the broader pricing survey is that the cheapest models are not automatically the cheapest useful coding models. If I restrict the list to APIs whose current first-party docs explicitly mention coding, code assistants, code completion, or software-engineering tasks, the lowest-cost credible floor is Command R7B from Cohere at \$0.0375 input and \$0.15 output per million tokens, followed by Mistral Small 4 at \$0.15 / \$0.60 and Codestral at \$0.30 / \$0.90. The cheapest explicit repo-agent model in the retrieved first-party docs is Devstral 2 at \$0.40 / \$2.00. The strongest mainstream daily-driver coding value is still GPT-5.4 mini from OpenAI, while the strongest current frontier-value challenger is Grok 4.3 from xAI.

The cleanest practical stack for your coding workflow is a three-lane setup. Use GPT-5.4 mini or Grok 4.3 for most real coding work, use Devstral 2 when you want a cheap repo-exploring agent that edits multiple files, and keep Claude Opus 4.7 or GPT-5.5 as escalators for the hardest debugging, architecture, and long-context reasoning jobs. If your goal is sheer spend minimization for repetitive code transforms, lint-style rewrites, or code-aware RAG assistants, Command R7B, Mistral Small 4, and Codestral are the current price-floor shortlist.

Async Pricing Can Flip the Rankings

One important operational caveat: asynchronous pricing can change the ranking materially. Batch and Flex pricing is available at half price for the GPT-5.5 / GPT-5.4 family; Batch and Flex halve Gemini 2.5 Pro and 2.5 Flash family prices; Anthropic batch processing gets a 50% discount on both input and output tokens; and xAI batch pricing is 20% to 50% below standard. If you run eval sweeps or nightly coding jobs, compare batch-adjusted cost, not only on-demand cost.

Cheapest Code-Capable APIs

The table below is sorted by a simple blended conversational proxy: $0.75 \times \text{input price} + 0.25 \times \text{output price}$, in USD per 1 million tokens. I intentionally excluded some ultra-cheap generalists, such as GPT-5.4 nano and Gemini 2.5 Flash-Lite, from the main shortlist because their current first-party docs emphasize simple high-volume work or budget multimodal usage rather than coding-specific positioning. They may still handle trivial scripting or transforms, but they are not clearly documented as code-first models.

#	Provider	Model	Official Coding Signal	In \$/M	Out \$/M	Blend	Context	Practical Lane
1	Cohere	Command R7B	"especially good" for code assistants	\$0.04	\$0.15	\$0.07	128k	Cheapest credible coding floor
2	Mistral AI	Mistral Small 4	"unifying instruct, reasoning, and coding"	\$0.15	\$0.60	\$0.26	256k	Cheapest general coding model
3	Mistral AI	Codestral	code completion, FIM, code generation	\$0.30	\$0.90	\$0.45	128k	Autocomplete, patching, FIM
4	Mistral AI	Devstral 2	"frontier code agents model"	\$0.40	\$2.00	\$0.80	256k	Repo agents, multi-file edits
5	xAI	Grok 4.3	xAI says use it for coding	\$1.25	\$2.50	\$1.56	1M	Frontier-value coding and tool use
6	OpenAI	GPT-5.4 mini	"strongest mini model yet for coding"	\$0.75	\$4.50	\$1.69	400k	Best mainstream daily default
7	Anthropic	Claude 4.5 Haiku	Not explicitly code-first; fast near-frontier	\$1.00	\$5.00	\$2.00	200k	Cheap Claude lane for code review
8	Mistral AI	Mistral Medium 3.5	optimized for agentic and coding use cases	\$1.50	\$7.50	\$3.00	256k	Higher-end open-weight coding
9	Google	Gemini 2.5 Pro	"excels at coding and complex reasoning"	\$1.25	\$10.00	\$3.44	1M	Google-native coding and long context
10	OpenAI	GPT-5.4	"more affordable model for coding"	\$2.50	\$15.00	\$5.63	1M	Heavier professional coding
11	Anthropic	Claude Opus 4.7	"most capable... for agentic coding"	\$5.00	\$25.00	\$10.00	1M	Premium coding and long-context debug
12	OpenAI	GPT-5.5	"flagship model for... coding"	\$5.00	\$30.00	\$11.25	1M	Premium coding; escalator not default

The price-floor story is clear. If you want the absolute cheapest documented coding-capable APIs, the current shortlist is Command R7B, Mistral Small 4, Codestral, and Devstral 2. If you want the cheapest frontier-like coding option, the best retrieved evidence points to Grok 4.3. If you want the safest everyday mainstream coding default with strong ecosystem support, the answer remains GPT-5.4 mini.

Standard Blended Cost — Explicit Code-Capable APIs

The chart below shows blended conversational cost for every explicitly code-documented model in the retrieved set, sorted cheapest to most expensive.

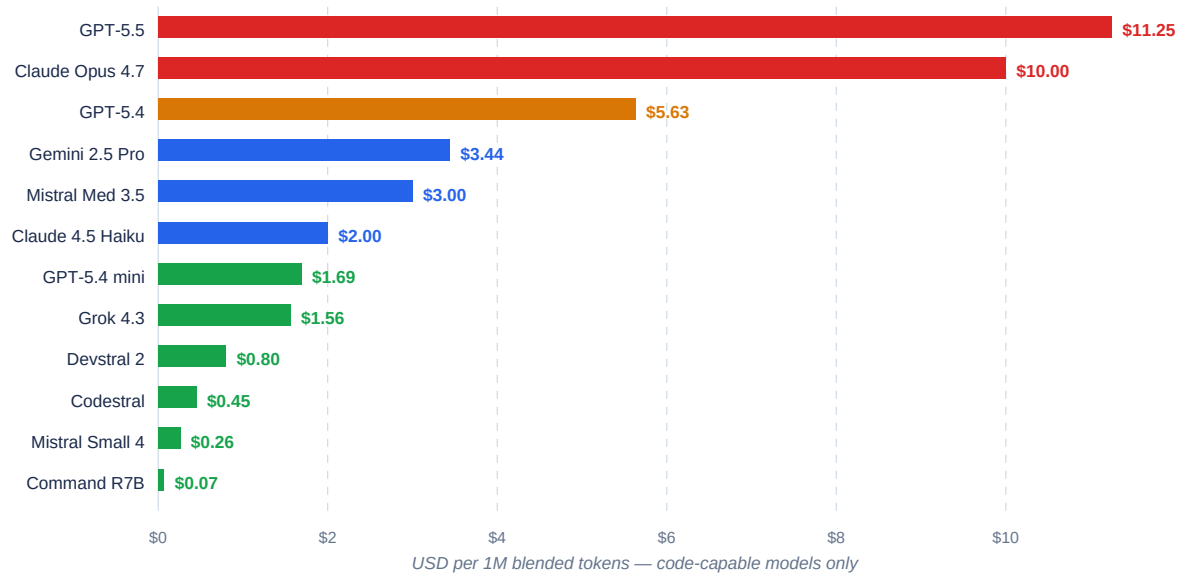


Figure 4. Blended cost for models with explicit code-first positioning in first-party docs. Green rows in table above correspond to green bars (under \$1/M).

A second-order insight is that providers differ in how cheap their coding story goes. OpenAI explicitly push GPT-5.5, GPT-5.4, and GPT-5.4 mini for coding, but not GPT-5.4 nano. Anthropic explicitly position only Opus 4.7 as the agentic-coding flagship, while Haiku 4.5 is the speed/volume tier. Google make Gemini 2.5 Pro the explicit coding model, while Flash and Flash-Lite are the value tiers. Cohere and Mistral are unusually direct and granular, giving you clear choices for completion, general coding, and repo-agent work.

Best Value by Provider

OpenAI

GPT-5.4 mini is the clear coding value pick. The docs call it the strongest mini model for coding, computer use, and subagents, and its standard pricing is already reasonable; Batch and Flex cut it in half again. GPT-5.5 is the premium escalator, and GPT-5.4 is the middle step when mini is not enough.

Anthropic

Claude Opus 4.7 is the coding flagship and Claude 4.5 Haiku is the budget throughput lane. That means Anthropic's own current documentation does not give you a cheap code-first model in the same way Mistral or Cohere do. Anthropic instead gives you price relief through batch discounting, prompt caching, and throughput gains from cache hits. For repeated coding contexts, that can narrow the real-world gap more than the rate card first suggests.

xAI

Grok 4.3 is the only new-build recommendation I would make right now. It is explicitly recommended for coding in the current xAI docs, and its pricing is unusually aggressive for a frontier-tier model. The cheaper grok-4-1-fast family remains visible in current pricing, but xAI says those older fast models retire on May 15, 2026, so they are not a sane foundation for a new evaluation harness on May 13, 2026.

Google (AI Studio and Vertex)

Gemini 2.5 Pro is the stable explicit coding choice in the retrieved official docs. Gemini 2.5 Flash is much cheaper and looks attractive for reasoning-heavy general work; Gemini 2.5 Flash-Lite is the baseline budget model. But Google's current stable docs reserve the explicit "coding capabilities" language for Pro, so that is the more evidence-based choice if you want to stay strict about code-first positioning.

Cohere

Command R7B is the cost floor and Command A is the step-up enterprise generalist. For coding specifically, R7B is the interesting one because the docs explicitly call out code assistants and give it very low price plus decent production rate limits.

Mistral AI

The coding lineup is the most differentiated. Codestral is the cheapest explicit completion model, Mistral Small 4 is the cheapest explicit general coding model, Devstral 2 is the cheapest repo-agent model, and Mistral Medium 3.5 is the higher-end open-weight coding option. That degree of specialization is unusual and makes Mistral the strongest provider to test if you want to discover the lowest price point that still clears your code quality floor.

Top Five Coding Recommendations for Your Keys

My top five coding recommendations for your keys are these.

#1	GPT-5.4 mini — Default Benchmark Anchor GPT-5.4 mini should be your default benchmark anchor because it is the strongest OpenAI mini for coding and gets much cheaper in Batch/Flex.
#2	Grok 4.3 — Frontier-Value Challenger Grok 4.3 should be your frontier-value challenger because xAI prices it much lower than premium incumbents while still positioning it as the coding choice.
#3	Devstral 2 — Specialist Repo-Agent Candidate Devstral 2 should be your specialist repo-agent candidate because it is explicitly built for software engineering workflows and multi-file edits at low cost.
#4	Command R7B — Ultra-Cheap Floor Test Command R7B should be your ultra-cheap floor test because it is the cheapest model in the official retrieved set that still makes a direct code-assistant claim.
#5	Claude Opus 4.7 — Premium Long-Context Escalator Claude Opus 4.7 should be your premium long-context escalator, especially if your coding workflow includes screenshots, documents, or repeated visual context where Anthropic's cache plus Files API combination matters.

Coding Decision Tree

Decision Criterion	Branch	Recommendation
Need the lowest possible coding cost?	Yes →	Command R7B or Mistral Small 4
Need code completion or FIM?	Yes →	Codestral
Need repo exploration and multi-file edits?	Yes →	Devstral 2
Need safest mainstream daily default?	Yes →	GPT-5.4 mini
Need best frontier-value ratio?	Yes →	Grok 4.3
Need maximum coding quality?	Yes →	Claude Opus 4.7 or GPT-5.5
Google-native long context and grounding	→	Use Gemini 2.5 Pro

Figure 5. Coding-specific decision tree. Evaluate top-to-bottom; stop at the first Yes branch.

Claude Prompt Caching for Uploaded Photos

Four Confirmed Points from Anthropic's Docs

The screenshots you shared line up with Anthropic's current docs. The docs confirm four points that matter for image-heavy debugging or screenshot-based coding workflows. First, Anthropic explicitly says images and documents in user turns can be cached. Second, Anthropic says the presence or absence of images anywhere in the prompt invalidates the message cache, so the cache only pays when the image set is stable across requests. Third, Anthropic's vision docs recommend uploading repeated images through the Files API and then referencing a `file_id` so you do not resend full image bytes on every turn. Fourth, Anthropic's pricing docs show cache writes at 1.25× base input price for 5 minutes or 2× for 1 hour, while cache hits cost 0.1× base input price, and those multipliers stack with batch pricing.

The practical implication is straightforward. If you repeatedly send the same screenshots, diagrams, or uploaded photos while asking new questions about them, Anthropic gives you two separate savings levers. The Files API reduces request payload size and network overhead. Prompt caching reduces repeated token billing and, on most models, reduces effective rate-limit pressure because cache reads do not count toward ITPM. If the screenshots change every turn, or if you add and remove images frequently, that advantage drops sharply because the message cache gets invalidated.

For your workflow, the right mental model is this: if you are doing repeated bug triage on the same UI screenshots, crash screenshots, or photos of printed output, Claude gets more economically interesting than its list price suggests. That is especially true for repeated compare/contrast or progressive questioning over the same images. The docs are explicit enough that I would treat “upload once with Files API, cache stable image blocks, ask many questions” as a first-class evaluation scenario in your bake-off. This is an inference from Anthropic's caching, rate-limit, and Files API guidance, not a vendor-published ROI guarantee.

Recommended Coding Bake-Off

The fastest way to settle this for your real use case is to run the same five test families across your keys and log exact cost, wall time, TTFT, sustained output speed, schema validity, test pass rate, and human preference. That will tell you more than another round of vendor positioning. The practical mistake to avoid is benchmarking only skill and not job cost. Batch pricing, cache hits, tool overhead, long context, and image reuse can radically change end-to-end economics.

Five Test Families

Test Family	What to Send	What to Score
Single-File Bug Fix	A failing test plus one source file. Require the smallest unified diff and a one-sentence root cause.	Tests passed, patch size, wall time, exact cost
Repo-Agent Task	A medium repo plus a goal that requires exploring files, changing multiple files, and summarizing risk.	Task completion, tool-use discipline, cost, latency
Screenshot-to-Code Debug	The same uploaded screenshot(s) in repeated turns with changing questions. Run once with raw image upload and once with Files API plus prompt caching on Claude.	Cost delta, payload size effect, answer consistency
Long-Context Coding RAG	50–200 pages of docs or manuals, then ask for exact API usage or implementation steps with citations or quoted evidence.	Retrieval accuracy, hallucination rate, output usefulness
Strict JSON Tool Call	Require exact JSON conforming to a schema for file edits, shell commands, or compile steps.	Schema validity, repair rate, retry count

Recommended Prompt Wording

For prompt wording, keep it brutally constrained. A good bug-fix prompt is: “You are fixing a real production bug. Return only a unified diff. Change as little as possible. If the test is underspecified, explain why in one sentence and stop.” A good repo-agent prompt is: “Explore the codebase, propose a plan, then edit only the files needed. Summarize changed files, risk, and rollback plan in under 120 words.” A good screenshot workflow prompt is: “Using only the attached image(s), identify the likely issue, list three hypotheses, and state which visual detail most strongly supports the top hypothesis.” A good long-context prompt is: “Answer only from the supplied documents. Quote the exact section or page evidence for each implementation step.” A good structured-output prompt is: “Return valid JSON matching this schema exactly. Use null when evidence is missing. Do not add prose.” These prompt forms are my recommended evaluation harness, not provider claims.

Open Questions and Limitations

I do not have a single apples-to-apples public coding benchmark number for every model in the shortlist from one current authoritative source. The official docs are uneven: some vendors make direct coding claims, some publish reasoning or tool-use positioning, and some do not expose a clean current coding score in the retrieved material. Because of that, I treated “explicit coding claim in first-party docs” as the primary filter for the cheapest shortlist, then used public benchmark context only sparingly where it was available and current enough to trust, such as Grok 4.3’s Artificial Analysis snippet.

The Google story deserves one caution flag. The retrieved stable Google docs make Gemini 2.5 Pro the explicit coding model, while Gemini 2.5 Flash and Flash-Lite are the obvious value tiers. That does not mean Flash or Flash-Lite cannot code; it means the current evidence I pulled is stronger for Pro as the official coding pick. If cheap Google-only coding is important to you, that provider is worth testing directly rather than inferring from marketing language alone.

The Anthropic image-caching conclusion is well supported, but its payoff depends heavily on your actual interaction pattern. If you reuse the same screenshots or uploaded photos across many turns, Anthropic’s Files API plus prompt caching should help materially. If every image changes, the benefit falls because image changes invalidate the message cache. That is the main reason I would include a dedicated screenshot-repeat benchmark in your evaluation matrix.